Colinha: ERP com Hibernate e DAO

Java Swing + Hibernate (JPA) + PostgreSQL/Supabase | UNIP - ALPOO - 4º trabalho

O sistema em uma frase
Programa desktop de **compra e venda**: cadastros, vendas, compras, controle de estoque, relatórios em PDF e dashboard. Os dados ficam no **PostgreSQL (Supabase)** e o acesso é feito com **Hibernate** através de classes **DAO**.

Tecnologias

Tecnologia	Para que serve no projeto
Java 17 + Swing (FlatLaf)	Telas, menus, tabelas; tema claro/escuro.
Hibernate ORM 6.6 (JPA)	Converte objetos Java em linhas do banco (e vice-versa). **Substituiu o JDBC manual.**
PostgreSQL no Supabase	Banco de dados na nuvem (11 tabelas do DER).
Maven (mvnw)	Baixa bibliotecas e gera o .jar. O wrapper dispensa instalar o Maven.
BCrypt / PDFBox / JFreeChart	Senha criptografada / relatórios PDF / gráficos do dashboard.

Camadas (MVC + DAO) - o caminho de uma ação

View Swing > **Controller** valida e delega > **Service** regras e transações > **DAO** consultas e CRUD > **Hibernate** objeto <-> SQL

Model (as entidades) atravessa todas as camadas. Cadastros simples (cliente, fornecedor, forma de pagamento) vão **Controller > DAO**. Operações que mexem em várias tabelas (**venda, compra**, login, produto) passam pelo **Service**, que controla a transação.

Pacotes em br.unip.erp

Pacote	Conteúdo
model	11 entidades anotadas: Pessoa, Cliente, Fornecedor, Usuario, Produto, FormaPagamento, Venda, VendaProduto, VendaPagamento, Compra, CompraProduto (+ DTOs de relatório/dashboard, que **não** são entidades).
dao	GenericDAO, JpaGenericDAO, DAOException e um DAO por entidade (+ DashboardDAO e RelatorioDAO).
service	VendaService, CompraService, UsuarioService (BCrypt), ProdutoService, AutorizacaoService, ReportService (PDF).
controller	Um por tela/módulo: valida os dados e chama DAO ou Service.
view	Telas Swing, históricos, dashboard (SwingWorker para não travar a tela).
util	JpaUtil (liga o Hibernate), Sessao, Permissao, Tema, Mascaras, ValidadorDocumento.

1. Hibernate / JPA - como liga tudo

Em uma frase
ORM (Mapeamento Objeto-Relacional): você trabalha com objetos (**Cliente, Venda**) e o Hibernate escreve o SQL. **JPA** é a especificação (pacote jakarta.persistence: anotações e interfaces); **Hibernate** é a implementação que o projeto usa.

As peças

Peça	Papel	Onde fica
persistence.xml	Declara a unidade **erpPU**: lista as 11 entidades, driver, dialeto PostgreSQL e o modo do schema.	src/main/resources/META-INF/
JpaUtil	Lê config.properties, config.local.properties e variáveis de ambiente e cria **uma única vez** o EntityManagerFactory.	util/JpaUtil
EntityManagerFactory	Fábrica "pesada" de EntityManager; existe 1 para o programa todo.	JpaUtil
EntityManager	Sessão de trabalho: persist, find, merge, remove, consultas. **Abre e fecha a cada operação.**	JpaUtil.getEntityManager()
EntityTransaction	begin / commit / rollback. Tudo ou nada.	DAO.transacionar e Services
JPQL	Linguagem de consulta com **nomes de classes e atributos** (não de tabelas).	dentro dos DAOs

```
<persistence-unit name="erpPU" transaction-type="RESOURCE_LOCAL">
  <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
  <class>br.unip.erp.model.Cliente</class>     <!-- ... mais 10 entidades -->
  <property name="hibernate.dialect" value="org.hibernate.dialect.PostgreSQLDialect"/>
  <property name="hibernate.hbm2ddl.auto" value="validate"/>
</persistence-unit>
```

hbm2ddl.auto - o que o Hibernate faz com as tabelas

Valor	Efeito	Uso
validate	Só confere se as tabelas batem com as entidades. Não cria nem apaga nada.	**Padrão do projeto**
none	Nem confere. Só usa o banco.	Local (db.ddl=none), se a validação reclamar
update / create / create-drop	Altera, recria ou apaga o schema automaticamente.	**Evitar** no banco real

Configuração da conexão (config.local.properties, na raiz; não vai para o GitHub)

```
db.url=jdbc:postgresql://aws-0-us-west-2.pooler.supabase.com:6543/postgres?sslmode=require&prepareThreshold=0
db.user=postgres.<codigo-do-projeto>
db.password=<senha do banco>
db.ddl=none
```

- **Porta 6543** = pooler do Supabase (PgBouncer): não aceita prepared statements no servidor, por isso **prepareThreshold=0**.
- O usuário usa o **código do projeto** (parte da URL do Supabase), não o nome que aparece no painel.
- Também funcionam as variáveis de ambiente DB_URL, DB_USER, DB_PASSWORD e DB_DDL.

2. Entidades e anotações (pacote model)

Anotação	Significa
@Entity / @Table(name="venda")	A classe é uma tabela do banco e qual é o nome dela.
@Id + @GeneratedValue(IDENTITY)	Chave primária; o **banco** gera o valor (coluna SERIAL).
@Column(name, nullable, length, precision, scale)	Mapeia o atributo para a coluna e define regras (tamanho, casas decimais).
@ManyToOne	Muitos para um (Venda > Cliente). O objeto tem a referência de um pai.
@OneToOne	Um para um (Cliente <-> Pessoa).
@OneToMany(mappedBy="venda")	Lista de filhos (itens da venda). **mappedBy** = lado inverso; a FK está no filho.
@JoinColumn(name="cli_codigo")	Nome da coluna de chave estrangeira.
cascade = ALL	Salvar/excluir o pai propaga para os filhos (salvar Venda grava itens e pagamentos).
orphanRemoval = true	Filho retirado da lista é apagado do banco.
fetch = LAZY	Relação só é carregada quando for acessada (ver armadilha abaixo).
@JdbcTypeCode(SqlTypes.CHAR)	Necessário nas colunas CHAR(1) (ativo, admin, tipo) para o **validate** aceitar no PostgreSQL.

```
@Entity @Table(name = "venda")
public class Venda {
  @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
  @Column(name = "vda_codigo")
  private Integer codigo;

  @ManyToOne(fetch = FetchType.LAZY, optional = false)
  @JoinColumn(name = "cli_codigo", nullable = false)
  private Cliente cliente;

  @OneToMany(mappedBy = "venda", cascade = CascadeType.ALL, orphanRemoval = true)
  private List<VendaProduto> itens = new ArrayList<>();      // lado INVERSO
}

// em VendaProduto:
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "vda_codigo", nullable = false)           // a FK mora aqui (lado DONO)
private Venda venda;
```

Relacionamentos do projeto

Relacionamento	Mapeamento
Cliente / Fornecedor <-> Pessoa	@OneToOne cascade ALL, FK pes_codigo única. Salvar o cliente salva a pessoa junto.
Venda > Usuario, Cliente	@ManyToOne LAZY.
Venda > itens e pagamentos	@OneToMany mappedBy + cascade ALL + orphanRemoval (VendaProduto, VendaPagamento).
Compra > Fornecedor, Usuario, itens	Igual: @ManyToOne para os pais e @OneToMany para CompraProduto.
Itens > Produto	@ManyToOne no VendaProduto / CompraProduto.

Detalhe do projeto
Os códigos são **Integer** (podem ser null), mas o **getCodigo()** devolve int e **0 quando for null**. Nas telas, **código 0 = registro novo** (insert); diferente de 0 = edição (update).

Armadilha: LazyInitializationException
Se a Venda sai do EntityManager (que já foi fechado) e alguém chama **venda.getCliente().getPessoa()** sem ter carregado, dá erro. A solução do projeto: os DAOs usam **join fetch** nas consultas (veja o VendaDAO na página seguinte).

3. DAO - o coração do trabalho

O que é
DAO (Data Access Object) é a camada que concentra **todo o acesso ao banco**. Telas e controllers chamam métodos como **inserir, listar, buscarPorCodigo** e não sabem nada de SQL, JPQL ou Hibernate.

- Separa a interface do banco: trocar consulta ou banco não mexe na tela.
- Reaproveita consultas em várias telas e deixa o código organizado e testável.
- Padroniza o CRUD: todos os DAOs têm os mesmos 5 métodos.

Hierarquia

```
GenericDAO<T, ID>            (interface: o contrato do CRUD)
   ^
JpaGenericDAO<T, ID>         (classe abstrata: implementa com EntityManager)
   ^
ClienteDAO, FornecedorDAO, UsuarioDAO, ProdutoDAO, FormaPagamentoDAO, PessoaDAO,
VendaDAO, CompraDAO          (cada um fixa o T e acrescenta consultas especificas)
```

T = a entidade (Cliente) e **ID** = tipo da chave (Integer). Ex.: `class ClienteDAO extends JpaGenericDAO<Cliente, Integer>`.

```
public interface GenericDAO<T, ID> {
  ID inserir(T entidade);   T atualizar(T entidade);   void excluir(ID id);
  T buscarPorCodigo(ID id); List<T> listar();
}
```

O que cada método faz no Hibernate

Método	Chamada JPA	SQL gerado
inserir	em.persist(e) + em.flush()	INSERT (devolve o id gerado)
atualizar	em.merge(e)	UPDATE (copia o estado do objeto "solto" para o gerenciado)
excluir	em.find(tipo, id) + em.remove(e)	DELETE
buscarPorCodigo	em.find(tipo, id)	SELECT ... WHERE id = ?
listar	JPQL "select e from Cliente e"	SELECT * FROM cliente

```
protected <R> R transacionar(Function<EntityManager, R> op) {   // usado por inserir/atualizar/excluir
  EntityManager em = JpaUtil.getEntityManager();
  EntityTransaction tx = em.getTransaction();
  try { tx.begin(); R r = op.apply(em); tx.commit(); return r; }
  catch (RuntimeException e) { if (tx.isActive()) tx.rollback(); throw erro("persistir", e); }
  finally { em.close(); }                                        // sempre fecha
}
```

Passos: abre o EntityManager > inicia a transação > executa > **commit**; se der erro, **rollback**; no fim, sempre **close**. Erros são embrulhados em **DAOException**. Leituras (listar/buscar) não precisam de transação: o try-with-resources já fecha.

DAO concreto: sobrescreve com join fetch

```
public class VendaDAO extends JpaGenericDAO<Venda, Integer> {
  public VendaDAO() { super(Venda.class); }

  @Override public List<Venda> listar() {
    try (EntityManager em = JpaUtil.getEntityManager()) {
      return em.createQuery(
        "select v from Venda v join fetch v.cliente c join fetch c.pessoa order by v.codigo desc",
        Venda.class).getResultList();     // 1 SELECT traz venda + cliente + pessoa (sem Lazy erro)
    }
  }
}
```

Outros DAOs especiais: **UsuarioDAO.buscarPorLogin** (login), **DashboardDAO** (SUM por mês com SQL nativo e totais em JPQL) e **RelatorioDAO** (consultas agregadas).

4. Ciclo de vida e transações (Service)

Estados de uma entidade

Estado	Como chega	O que acontece
Transient (nova)	new Cliente()	O Hibernate ainda não conhece o objeto.
Managed (gerenciada)	persist() ou find()	Dentro do EntityManager aberto. **Mudanças são gravadas sozinhas no commit** (dirty checking).
Detached (solta)	em.close()	Objeto ainda existe, mas o Hibernate não o acompanha mais. Para gravar de novo: **merge()**.
Removed	remove()	Será excluída no commit.

persist x merge
persist: insere um objeto novo. **merge**: atualiza a partir de um objeto solto (vindo da tela) e devolve a cópia gerenciada. O `JpaGenericDAO` usa persist em inserir e merge em atualizar.

Venda: tudo em uma transação (VendaService.registrar)

1. **Valida**: cliente, ao menos 1 item, quantidades e preços positivos, e **soma dos pagamentos = total**.
2. Abre o EntityManager e **tx.begin()**.
3. Busca usuário, cliente e formas de pagamento e confere se estão **ativos**.
4. Para cada produto: **em.find(..., PESSIMISTIC_WRITE)** (SELECT ... FOR UPDATE) trava a linha para ninguém vender o mesmo estoque ao mesmo tempo. Os produtos são travados em **ordem de código** para evitar deadlock.
5. Confere se há **estoque suficiente** e baixa o estoque (sem chamar update: o dirty checking faz no commit).
6. **em.persist(venda)**: o cascade grava os itens e os pagamentos.
7. **tx.commit()**. Qualquer erro > **rollback**: nada é gravado. No **finally**, em.close().

```
tx.begin();
Produto p = em.find(Produto.class, id, LockModeType.PESSIMISTIC_WRITE);  // trava a linha
if (p.getEstoque().compareTo(qtde) < 0) throw new IllegalArgumentException("Estoque insuficiente");
p.setEstoque(p.getEstoque().subtract(qtde));    // dirty checking: UPDATE automatico no commit
em.persist(venda);                              // cascade: grava itens e pagamentos
tx.commit();                                    // erro em qualquer ponto -> rollback
```

Operação	Efeito no estoque
Registrar venda	Baixa a quantidade vendida (e exige estoque suficiente).
Registrar compra	Soma a quantidade comprada.
Excluir venda / compra	Devolve / retira o estoque na mesma transação. **Só administrador** (AutorizacaoService.exigirAdmin relê o usuário no banco).

Por que Service e não só DAO?
O **JpaGenericDAO** abre uma transação por operação. A venda mexe em **4 tabelas** (venda, itens, pagamentos, produto) e precisa ser **uma única transação**; por isso a regra e a transação ficam no Service.

5. Outras peças, erros comuns e perguntas de prova

Tema	Resumo
Login e segurança	Senha com **BCrypt**. Senha antiga em texto é convertida no 1º login. "Criar usuário" no login cria sempre usuário **comum**. Excluir = só admin. Login inicial: **admin / admin**.
Relatórios PDF	ReportService (PDFBox): 7 relatórios (usuários, clientes, fornecedores, produtos, formas de pagamento, vendas, compras). Botão "Relatório PDF" em cada tela; arquivos na pasta relatorios/.
Dashboard	Abre após o login: faturamento, compras, saldo, itens abaixo do mínimo, linha vendas x compras (12 meses) e anel por forma de pagamento. Carrega com SwingWorker.
Banco de dados	**schema.sql** apaga e recria tudo (só para banco novo). **migration_v2_hibernate.sql** atualiza o banco existente sem apagar dados.
Executar	Java 17+. `executar.bat` (Windows) ou `./executar.sh`; ou `mvn clean package` e `java -jar target/sistema-erp-hibernate.jar`.

Erros comuns

Sintoma	Causa provável	Solução
Unable to build SessionFactory	URL/senha errada ou tabelas não batem com as entidades.	Ler a linha "Caused by" no console; conferir config.local.properties; testar db.ddl=none.
tenant/user not found	Host ou usuário do pooler errado.	Usuário postgres.<codigo-do-projeto> e host do pooler do Supabase.
password authentication failed	Senha do banco incorreta.	Conferir ou redefinir a senha em Database Settings do Supabase.
LazyInitializationException	Acessou relação LAZY com o EntityManager fechado.	Usar join fetch na consulta do DAO.
Removing a detached instance	Reusou um objeto já salvo (ex.: mesma Venda).	Criar uma **nova instância** após finalizar (como faz o botão Nova).

Perguntas que podem cair

Pergunta	Resposta curta
Diferença entre JPA e Hibernate?	JPA é a especificação (anotações/interfaces); Hibernate é a implementação usada.
O que o DAO resolve?	Isola o acesso ao banco: tela e regra não conhecem SQL/Hibernate; troca de banco ou consulta não afeta a tela.
Para que serve o GenericDAO<T, ID>?	Contrato único de CRUD reaproveitado por todas as entidades, evitando código repetido.
persist ou merge?	persist insere objeto novo; merge atualiza a partir de objeto solto.
O que fazem cascade e orphanRemoval?	cascade propaga salvar/excluir ao filho; orphanRemoval apaga o filho retirado da lista.
O que é mappedBy?	Indica o lado inverso: a FK está na tabela do filho, que tem o @ManyToOne.
Por que hbm2ddl = validate?	Para o Hibernate só conferir o schema e nunca criar ou apagar tabelas no banco real.
Como evita vender mais que o estoque?	Lock pessimista (FOR UPDATE) no produto + conferência de saldo dentro da mesma transação.
O que é JPQL?	Consulta orientada a objetos (classes e atributos), traduzida para SQL pelo Hibernate.
Onde fica o MVC?	Model = entidades; View = Swing; Controller = valida e delega; DAO/Service = acesso a dados e regras.